

Reti di Calcolatori

Nicolò Giuliani

Indice

1	Cosa è una rete di calcolatori?	3
1.1	Canali di comunicazione	5
1.2	Protocolli di rete organizzati a livelli	6
2	Reti di reti e Internetworking	8
2.1	Internet protocol (IP)	8
2.2	Ipv4	8
2.3	Subnetwork	9
2.4	Forwarding	9
2.5	Routing	9
2.6	Protocollo ICMP	10
2.7	ARP e RARP	10
2.8	DHCP	11
2.9	IPv6 e tunnelling IPv4	11
2.10	Livello trasporto: TCP / UDP	11
2.11	Controllo di flusso e congestione	12
2.12	DNS e nomi di dominio	14
2.13	Livello applicazione	14
2.14	Servizi differenziati e Internet2	15
3	Sicurezza di rete	15
3.1	RSA	16
3.2	Autenticazione	18
3.2.1	Digital signatures	18
3.2.2	Certification authority	19
3.3	Securing e-mail	19
3.4	Securing TCP connections: SSL	20
3.4.1	Real SSL	22
3.5	Network layer security: IPsec	23
3.5.1	VPN	23
3.6	Securing wireless LANs	25
3.7	Operational security: firewalls and IDS	26
3.7.1	Firewall	26
3.7.2	IDS	27
4	Data Plane	27
4.1	Network layer	27
4.1.1	Data plane	27
4.1.2	Control plane	28
4.2	Come e' fatto un router	28

4.2.1	Input	29
4.2.2	Output	31
4.3	Internet Protocol	32
4.3.1	Datagram format	32
4.3.2	Fragmentation	32
4.3.3	IPv4 addressing	33
4.3.4	Network address translation	35
4.3.5	IPv6	35
4.4	Generalized Forward and SDN	36
4.4.1	OpenFlow data plane abstraction	37

5 Control Plane 37

1 Cosa è una rete di calcolatori?

E' un insieme di calcolatori interconnessi tra loro per la comunicazione fra utenti, informazioni, risorse, ecc.

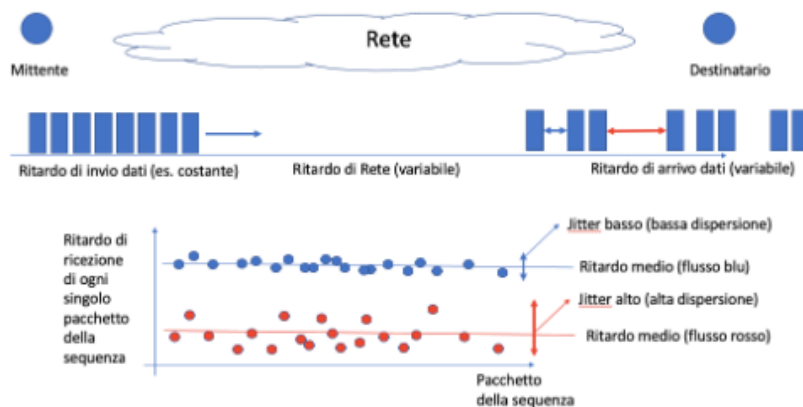
Classificazione

- Reti locali (LAN)
- Reti geografiche (WAN)
- Reti metropolitane (MAN)

Definiamo con **internet** la rete globale delle varie reti connesse tra loro via protocolli.

Prestazioni

Misuriamo la velocità di una rete con i **bit** (b/s) o byte (B/s), ovvero gruppo da 8bit. Definiamo **ping** la latenza tra la partenza totale del pacchetto al suo arrivo dal destinatario, questo è data dalla tipologia di tecnologia utilizzata, protocolli e distanza fisica. Chiamiamo **jitter** la varianza media nel tempo di arrivo al destinatario dei pacchetti.



Come ci si connette?

Abbiamo bisogno di un insieme di componenti hardware e software:

- **Scheda di rete:** trasmettere, codificare, decodificare i segnali elettrici in bit digitali.
- **Driver:** software che gestisce il dispositivo fisico con il S.O.
- **Connettore (fisico) di rete:** connettore standard della scheda di rete.
- **Mezzo di trasmissione:** fibre ottiche, cavi rame, ponti radio, ecc.
- **Protocolli:** regole via software per avere una piena compatibilità su tutti i dispositivi.

I vari calcolatori connessi in rete vengono divisi in *host* e *node*. Dividiamo in 2 macro categorie le infrastrutture di reti:

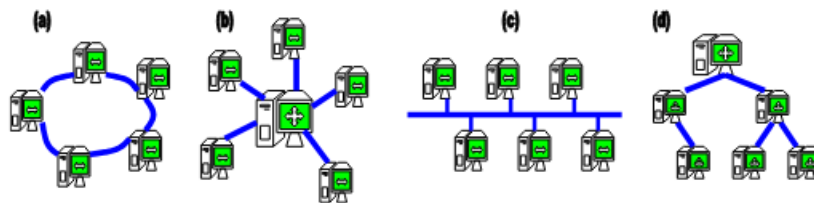
- Peer to peer (P2P)
- Strutture a connessioni multiple, con cammino di segnali diretto o in sequenza.

Topologia

Abbiamo come standard diverse schematiche di reti per connettere dei calcolatori:

- a) Anello
- b) Stella
- c) Bus
- d) Albero

Ovviamente le reti più grandi utilizzano una struttura mista.



Curiosità: In Italia abbiamo la rete **GARR** ovvero la rete italiana a banda ultralarga dedicata alla comunità dell'istruzione, della ricerca e della cultura.

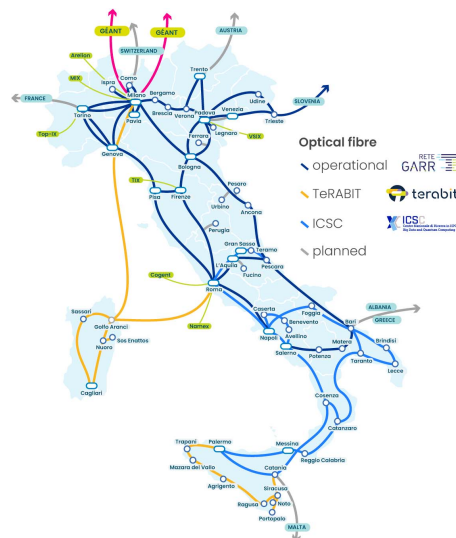


Figura 1: Mappa aggiornata a luglio 2024

Mezzo fisico di trasmissione

Abbiamo 3 tipi di mezzo di trasmissioni:

- Cavi / Fili metallici: segnali elettrici (corrente e variazione di tensione)
- Fibre ottiche: segnali luminosi dentro fibre di vetro
- Wireless: radiazioni elettromagnetiche nello spazio come onde radio e infrarossi

C'è anche una sostanziosa differenza tra hub e switch, nella topologia a stella! L'hub (lavora in L1) "spara" il segnale elettrico in ogni porta e quindi capita più frequentemente collisione dei pacchetti (e la loro perdita). Costavano meno, e si "pregava" di più nel protocollo che sistemasse i pacchetti persi; andava bene per reti ridotte. Ad oggi gli switch che hanno preso la prevalenza nel mercato, anche a basso costo, ci permettono di minimizzare i conflitti reinstradando i pacchetti, lavora al L2 e basandosi sul MAC address del dispositivo connesso ad una porta reindirizza il pacchetto SOLO su quella porta. Mentre un router che lavora con indirizzi IP lavora ad L3.

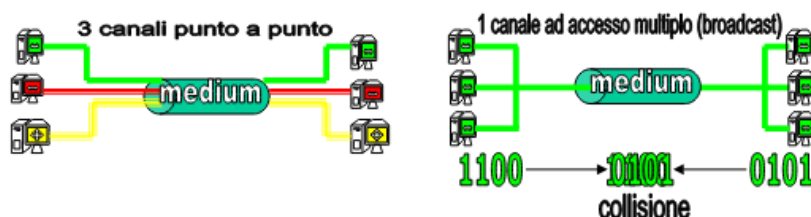
Scheda di rete

Componente del calcolatore che ha il connettore di rete e l'interfaccia tra scheda e mobo. Attraverso il proprio processore, memoria, chip, condensatori, ecc la scheda riceve/manda e codifica/decodifica i segnali elettrici/digitali. Il suo codice identificativo unico si chiama **MAC ADDRESS** (medium access control).

La scheda per comunicare con il calcolatore e il SO ha bisogno di un applicativo software che gestisca correttamente questa comunicazione, questo è il *driver*.

1.1 Canali di comunicazione

Come abbiamo visto prima abbiamo P2P o canale broadcast. Soffermiamoci sul secondo, tutti i dispositivi comunicano nello stesso canale come siamo sicuri che non si trasmette/riceve nello stesso momento coordinando tutti? Attraverso l'indirizzo univoco del destinatario (MAC) e del mittente (MAC).

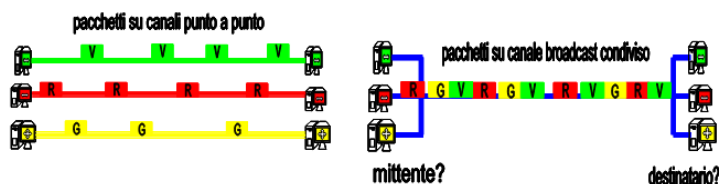


Tecnica di comunicazione

- **Commutazione a circuito** Prima che due dispositivi inizino a comunicare viene stabilito il percorso (circuito dedicato) dalla sorgente al destinatario, per la durata della comunicazione il canale rimane riservato ed al termine viene liberato. Questo offre una buona velocità ma con alto ritardo e poco costante in caso di pause, tutto questo a costo per tempo. Un esempio è la rete telefonica.
- **Commutazione a pacchetto** Standard per Internet e le comunicazioni broadcast. Tutti i dati vengono suddivisi in pacchetti indipendenti, spediti su canali diversi. Questo modello si può utilizzare sia per connessioni P2P che per le broadcast. Nel secondo caso dobbiamo definire anche: l'indirizzo del destinatario per ogni pacchetto, i pacchetti viaggiano tra flussi diversi di pacchetti. Questo offre una ottimizzazione delle risorse della rete con a costo però maggior ritardo nella comunicazione. Si paga per quantità di dati.

Componendo in serie una sequenza di canali broadcast a commutazione di pacchetto, i nodi ricevitori devono di volta in volta farsi carico di verificare se il pacchetto sia giunto a destinazione o, in caso contrario, possono provvedere all'inoltro del pacchetto ricevuto sul successivo canale

broadcast. Il prezzo di questa tecnica è legato al maggiore ritardo per la comunicazione dei dati tra mittente e destinatario, dovuto all'esigenza di iterare più volte la ricezione e l'inoltro di pacchetti su canali broadcast in sequenza. Il vantaggio è legato al maggiore utilizzo dei canali ad accesso multiplo, e quindi alla possibilità di tariffare la comunicazione in base ai dati trasmessi, e non in base al tempo necessario.



1.2 Protocolli di rete organizzati a livelli

Un protocollo è un insieme di regole e procedure di gestione dei processi di comunicazione. Quindi si è definita una *architettura* di rete divisa in livelli nel quale ognuno affronta problemi diversi, dal basso al alto si forniscono servizi. Questo insieme crea il protocollo per l'interfaccia di rete.

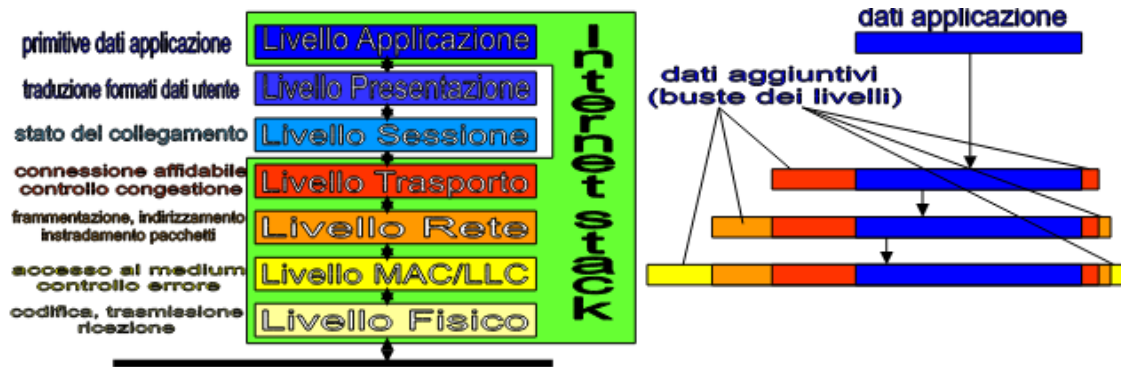
Nel 1984 si è ufficializzato lo standard **ISO/OSI** (Open System Interconnection Reference Model), la struttura di rete è divisa in 7 livelli nel quale più ci troviamo in alto più il modello di rete si semplifica.



L'architettura moderna di Internet utilizza solo 5 livelli del modello ISO/OSI, ogni livello riceve dati dai livelli superiori e li inserisce (incapsula) in "buste" con dati aggiuntivi, utili ad istruire il corrispondente livello del dispositivo ricevente.

- Il livello trasporto imbusta i dati aggiungendo informazioni utili all'ordinamento e controllo della velocità di invio delle buste
- Il livello rete frammenta i dati in pacchetti, decide il cammino sul quale inviare il pacchetto a seconda dell'indirizzo del destinatario
- Il livello MAC/LLC esegue la consegna finale dei dati a dispositivi di una rete locale

Vediamo ora nel dettaglio i 5 livelli del TCP/IP:



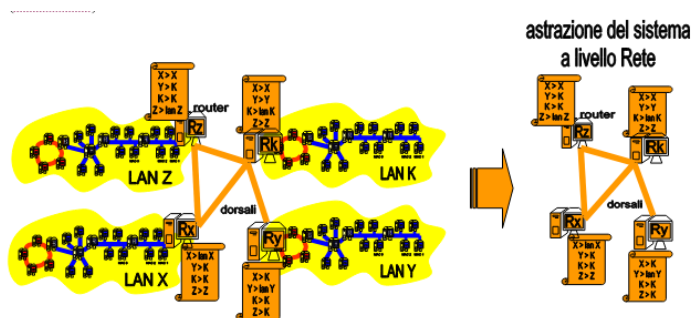
- 1) livello fisico
 - trasmissione sul canale di comunicazione fisicamente attraverso segnali analogici, derivanti da codifica dei segnali in bit. La velocità dei segnali analogici è pressochè quella della luce, si misura però in bit/secondo + tempo di decodifica da analogico → digitale
- 2) livello MAC/LLC
 - rete locale (LAN). Frame.
- 3) livello rete
 - attraverso i router colleghiamo le LAN tra di loro, qui nasce internet. Pacchetti.
- 4) livello trasporto
 - garantire che i dati arrivino correttamente, in ordine e senza duplicati, tra due processi tra due macchine diverse (TCP/UDP).
- 5) livello applicazione
 - servizio in utilizzo, Dati (HTTP, FTP, SSH, ...)

Come si è sicuri che un frame sia arrivato a destinazione? Se ne occupa il LLC con il rilevamento degli errori, controllo del flusso e conferma di ricezione:

- Sì! il pacchetto arrivato a destinazione, il destinatario invia un frame di conferma (ACK)
- No! il mittente non ricevendo la conferma trasmette di nuovo il frame

2 Reti di reti e Internetworking

Le reti locali sono connesse attraverso collegamenti organizzati in modo gerarchico, basati su calcolatori rappresentanti della rete locale (router) a loro volta collegati da linee dati veloci o dorsali (backbone).



2.1 Internet protocol (IP)

Il concetto di rete non si limita ora alla sola rete locale (LAN) ma si estende alla rete di reti globale (Internet).

- Internet:
 - nuovo tipo di indirizzamento globale e gerarchico (indirizzamento IP)
 - instradamento dei pacchetti dal mittente al destinatario finale (forwarding)
 - nuovi dispositivi amministratori del livello tre: router
 - frammentazione dei dati da spedire in pacchetti
 - busta del pacchetto di livello rete con gli indirizzi di mittente e destinatario

2.2 Ipv4

Un indirizzo IP viene associato a una e una sola interfaccia di rete (scheda di rete) che a sua volta ha una associazione univoca tra indirizzo MAC.

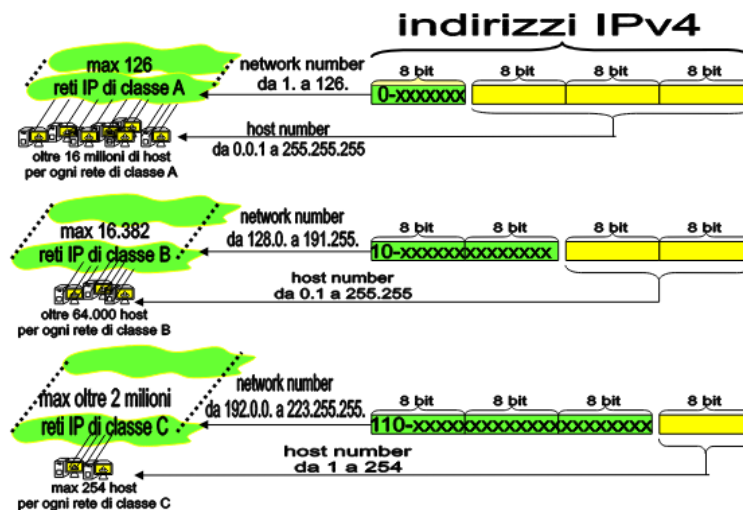
Vediamo le caratteristiche

$$\text{Ipv4} = 4\text{byte} (32\text{bit})$$
$$\text{xxx.xxx.xxx.xxx (da 0 a 255)}$$

Indirizzo IP è sempre composto da due parti:

- Numero della rete IP alla quale appartiene la scheda (**network number**)
- Numero dell'interfaccia di rete (**host number**) all'interno della rete

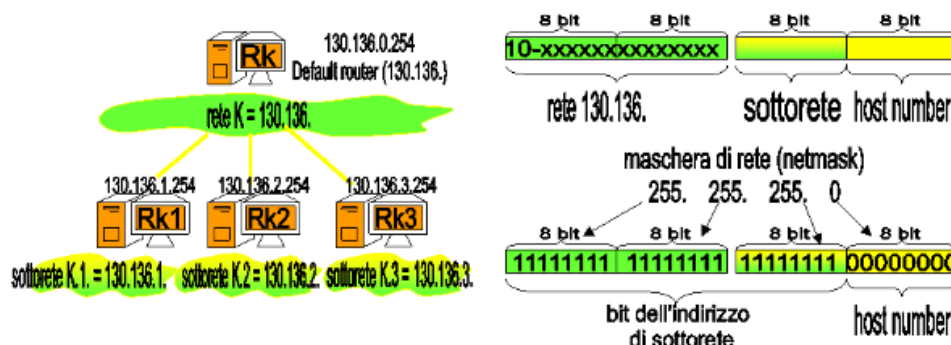
Il valore dell'indirizzo IP determina la **classe** della rete: A,B,C (non si usano più).



2.3 Subnetwork

Reti IP e router sono organizzati secondo una gerarchia, basata sul concetto di rete e sottorete.

- indirizzo IP spezzato in due componenti logiche mediante maschera di rete (netmask)
 - indirizzo di sottorete (subnetwork)
 - host number dell'host appartenente alla sottorete



Indirizzo IP, netmask e default router sono informazioni di configurazione del livello rete

2.4 Forwarding

Processo con cui un dispositivo di rete, come un router, riceve un pacchetto dati da una rete e lo invia verso un'altra rete, seguendo le informazioni contenute nell'header del pacchetto e nella tabella di routing del dispositivo.

2.5 Routing

Processo mediante il quale un dispositivo di rete, tipicamente un router, decide il percorso migliore che un pacchetto dati deve seguire per arrivare dalla sorgente alla destinazione.

Aggiornamento delle tabelle di forwarding dei router: algoritmi di routing su Internet: Routing Information Protocol (RIP), Open Shortest Path First (OSPF), Border Gateway protocol (BGP).

2.6 Protocollo ICMP

Internet Control Message Protocol (ICMP); protocollo dei messaggi di controllo su Internet.

- Standard per definire la comunicazione di informazioni utili alla gestione di Internet
- usato da host, router e gateway per scambiare informazioni di livello rete, usando pacchetti definiti con il protocollo IP

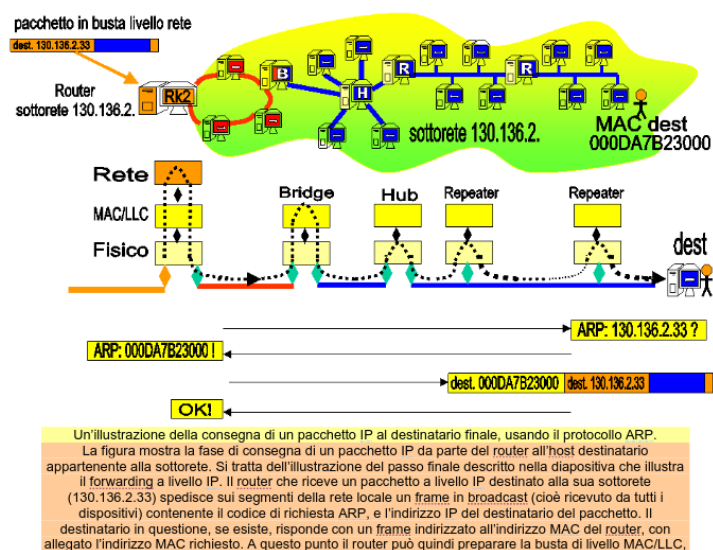
Vediamo delle applicazioni basate su ICMP

- *Ping*: test di verifica di connessione tra due host
- *Traceroute*: mostra la sequenza di router attraversati da un pacchetto per arrivare all'host destinazione

2.7 ARP e RARP

Quando un router riceve un pacchetto destinato a un indirizzo IP della propria sottorete, esso deve produrre un frame di livello MAC/LLC che riporti specificato l'indirizzo MAC del destinatario (e non l'indirizzo IP), per poterlo passare al livello MAC/LLC per la trasmissione sulla rete locale.

- Address Resolution Protocol (ARP):
 - Usato se il router non conosce l'indirizzo MAC corrispondente all'indirizzo IP
 - Il router genera un frame spedito a tutti i dispositivi della rete locale dove si chiede: “quale è l'indirizzo MAC del dispositivo che ha questo indirizzo IP”?
 - Se tale dispositivo esiste, esso risponde con un frame di livello MAC indirizzato al router, nel quale viene evidenziato l'indirizzo MAC richiesto
- Reverse-ARP (RARP):
 - Domanda è “quale indirizzo IP corrisponde al dispositivo con questo indirizzo MAC”? (inverso di ARP)



2.8 DHCP

- Assegnazione dei numeri di rete di classe A, B, C
 - vengono assegnati a enti, aziende ed organizzazioni che ne fanno richiesta, da parte di enti internazionali (RIPE, ICANN, ARIN, APNIC)
- Assegnazione dei numeri di host ai dispositivi di una rete
 - Assegnazione manuale
 - Assegnazione automatica (**DHCP**)
 - * Un server Dynamic Host Configuration Protocol si occupa dell'assegno dell'indirizzo IP agli host che ne fanno richiesta
 - * Un DHCP server dispone di un blocco di host number liberi per la sua rete
 - * DHCP server associa indirizzi IP a indirizzi MAC dei dispositivi che lo richiedono
 - * I dispositivi devono scegliere di affidarsi al server DHCP

2.9 IPv6 e tunnelling IPv4

Dal 1990 e' attivo IPv6, indirizzi estesi a 128 bit (16 Byte) (prima erano a 32bit).

Tunneling: spedire pacchetti IPv6 in buste IPv4.

2.10 Livello trasporto: TCP / UDP

I protocolli di Internet di quarto livello (trasporto) sono essenzialmente il Transmission Control Protocol (TCP) e lo User Data Protocol (UDP).

- Servizio trasporto affidabile (protocollo **TCP**): servizio di tipo connection-oriented
 - numero di porta del socket TCP
 - eliminazione dei pacchetti duplicati
 - Pacchetti conferma della ricezione (acknowledgment di livello trasporto) + Pacchetti non ricevuti (non confermati entro timeout) spediti di nuovo
- Servizio trasporto non affidabile (protocollo **UDP**): servizio connectionless



TCP

E' il protocollo di livello trasporto usato di fatto su Internet (TCP/IP).

- Socket (indirizzo ip + porta del livello superiore)

- applicazioni in attesa sul socket
- client invia richiesta TCP al socket server
- server risponde "ok" (se il socket del server esiste e non e' occupato)
- il client (se riceve "ok") puo' iniziare a inviare i dati di configurazione TCP
- ora avviene il vero scambio a livello TCP
- rilascio della connessione (si liberano le porte)

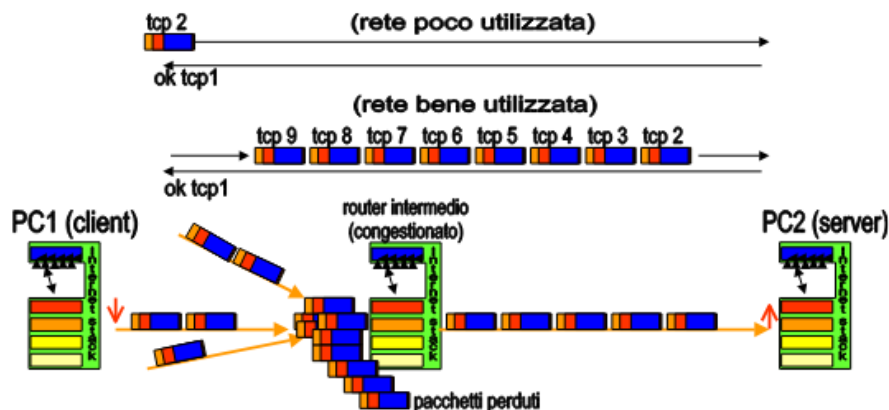


2.11 Controllo di flusso e congestione

Problema: la latenza di rete (tempo di andata e ritorno dei pacchetti) è alta

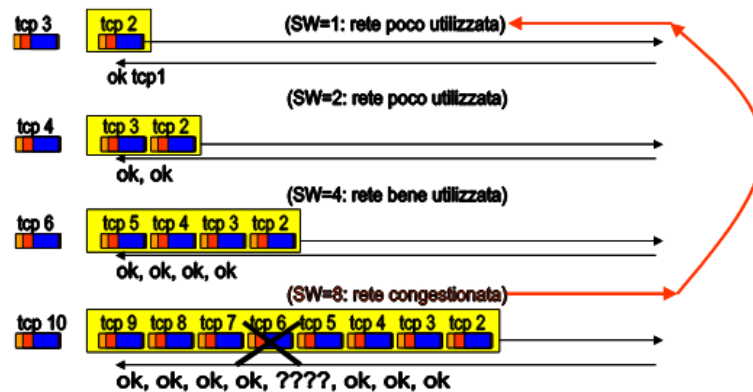
- Invio pacchetto e attendo conferma prima di inviare il successivo?
 - Solo un pacchetto in tutto il cammino: troppo lento!!!
- Invio tutti i pacchetti uno di seguito all'altro in un colpo solo?
 - Possibile congestione nei router intermedi, che perdono pacchetti: troppo veloce!
 - Possibile saturazione del destinatario, che riceve pacchetti troppo velocemente!
- Scopo del **controllo di flusso**: inviare pacchetti al massimo ritmo sostenibile dal destinatario finale
- Scopo del **controllo di congestione**: inviare pacchetti al massimo ritmo sostenibile dal router più lento della rete dal mittente al destinatario finale

Esistono vari metodi: meccanismo a **finestra scorrevole** (sliding window, SW).



Una finestra scorrevole (sliding window, SW) è un valore intero, (es. $n \in \mathbb{N}$) e rappresenta il numero massimo di pacchetti che un mittente può spedire di seguito, in attesa di ricevere la loro conferma.

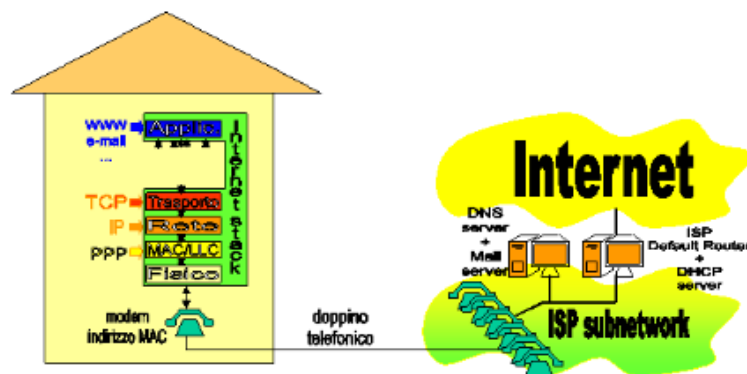
- Ogni mittente TCP spedisce al massimo ritmo possibile un gruppo di SW pacchetti
- **Controllo di flusso:** non spedisce più di SW pacchetti oltre l'ultimo non confermato
- **Controllo della congestione:** spedisce un numero SW variabile di pacchetti
 - Se SW pacchetti spediti sono tutti confermati, aumenta SW ($2 \rightarrow 4 \rightarrow 8 \rightarrow \dots$)
 - Appena un pacchetto degli SW inviati non viene confermato (scade il timeout)
 - * Assume che ciò sia dovuto a congestione su un router
 - * Riparte da SW minimo
 - Cerca di accelerare il ritmo gradualmente e, appena scopre la congestione, rallenta.



TCP/IP

Cosa serve per configurare un host per la connessione a Internet?

- Installare il dispositivo o scheda di rete
 - Creare istanza dello stack TCP/IP e PPP per il dispositivo (modem)
 - (Installare firewall per impedire accessi esterni all'host?)
 - * Bloccare l'accesso ai dati e alle applicazioni degli intrusi
 - Filtrare i pacchetti a livello rete
 - Firewall: è il primo router visto dall'esterno della rete, che filtra i pacchetti
- Informazioni da fornire (manualmente, o attraverso DHCP server dell'ISP)

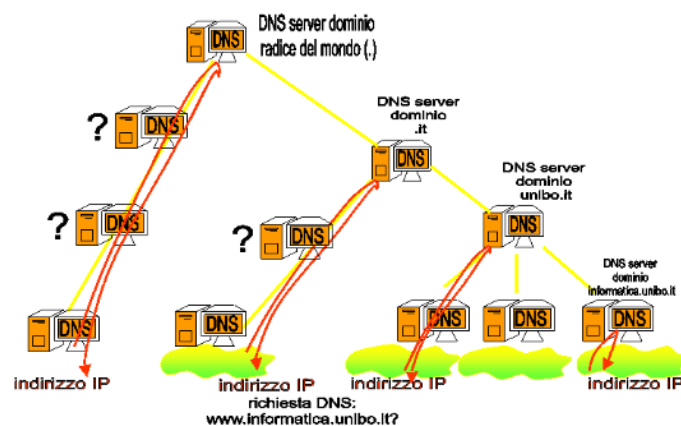


2.12 DNS e nomi di dominio

Gli utenti preferiscono usare nomi per le risorse in rete, anziché indirizzi IP.

Problema: I protocolli di rete e i router pretendono di usare indirizzi IP.

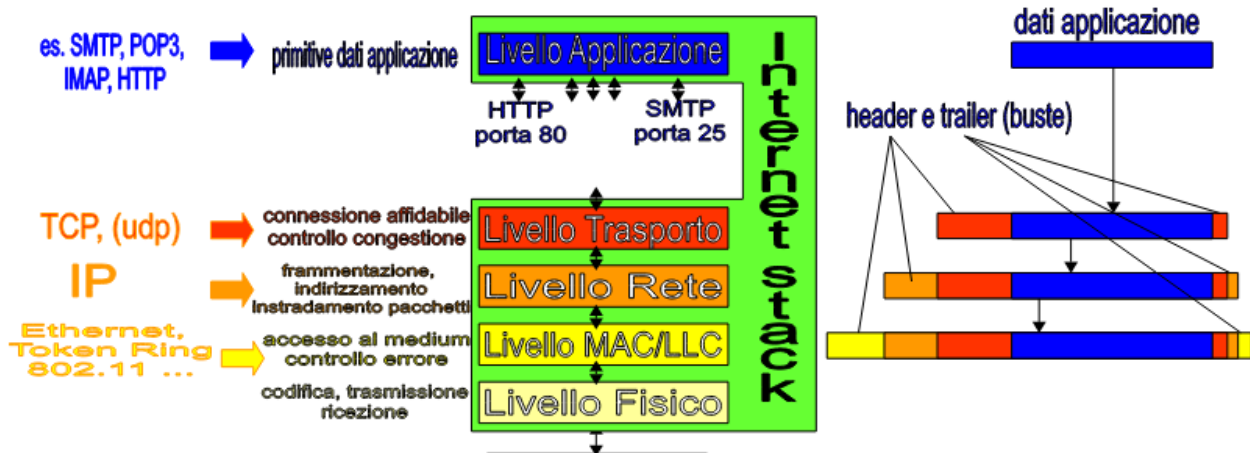
- Domain Name System (DNS)
 - Basato su una catena di server DNS organizzati gerarchicamente
 - * Ogni host in rete deve conoscere almeno un DNS server
 - * Ogni server DNS conosce almeno un DNS server superiore
 - I server ricevono richieste (protocollo DNS) e forniscono indirizzi IP
 - Se un server non conosce la risposta inoltra la richiesta DNS a un server superiore
 - I server DNS radice (DNS root server) conoscono tutti i domini e i loro IP! (ma sono pochi e costosi)



2.13 Livello applicazione

Il livello applicazione (si appoggia sul livello trasporto (in particolare sul protocollo TCP)): primitive e protocolli per spedire e ricevere dati delle applicazioni.

- Posta elettronica (SMTP, POP3, IMAP)
- World Wide Web (HTTP)
- DNS: Domain Name Service (protocollo DNS)



Servizi Client/Server e Peer to Peer

Le applicazioni e i servizi su Internet possono essere realizzati secondo due modalità architettureali

- Architettura Client/Server
- Architettura Peer to Peer (P2P)
- Servizi ibridi

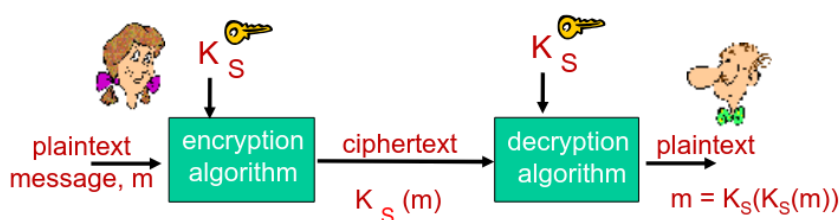
2.14 Servizi differenziati e Internet2

Un aspetto critico della comunicazione su Internet è la mancanza di garanzie sui tempi di consegna dei dati (qualità del servizio).

- Posso imporre che i pacchetti arrivino tutti (servizio connection-oriented di TCP)
- Non posso imporre che i pacchetti arrivino entro tempi fissati su Internet
 - Tutti i pacchetti sono distribuiti con la stessa urgenza
- Internet2
 - Nuove linee dorsali e nuovi router
 - I router spediscono prima i pacchetti urgenti, e poi quelli non urgenti
 - Riservare le risorse lungo il cammino per comunicare i dati
 - Garantisce la qualità del servizio su tutto il collegamento mittente-destinatario

3 Sicurezza di rete

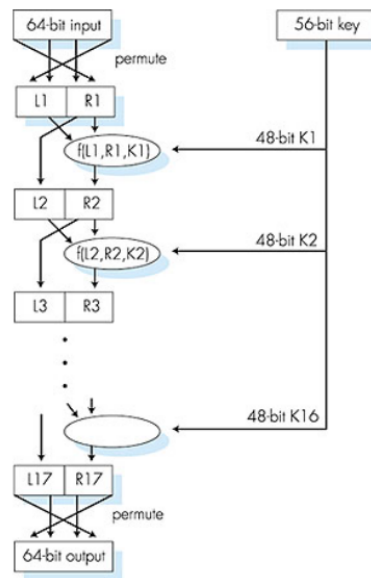
Alla base troviamo la crittografia con chiave condivisa su testo attraverso algoritmo in chiaro.



Come ci mettiamo d'accordo sulla chiave comune?

- Substitution cipher: sostituiamo una lettera dell'alfabeto con un'altra.
→ *Encryption key*: mappiamo 26 lettere con altre 26 lettere.
- n -substitution ciphers: ciclamo n cifrai ad ogni lettera.
→ *Encryption key*: ciclare n -cifrai di 26 lettere.

- DES (Data Encryption Standard): input text di 64bit, chiave di 56bit. 16 round di funzione su 48bit della chiave.



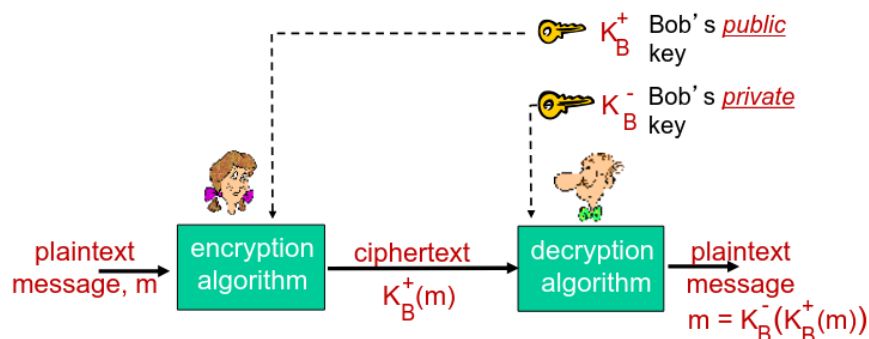
- AES (Advanced Encryption Standard): rimpiazzo del DES, chiave da 128/192/256 bit, 10/12/14 round.

Abbiamo detto che nella **symmetric key crypto** il sender e receiver devono condividere la chiave segreta. Ma non è possibile avere una chiave senza una comunicazione insicura precedentemente. Come risolviamo?

Public key crypto Non abbiamo sender e receiver con una chiave condivisa, ma

- *public encryption key*: condivisa a tutti K_B^-
- *private decryption key*: solo receiver K_B^+

$$\text{msg} = K_B^-(K_B^+)$$



3.1 RSA

Così abbiamo ottenuto *RSA* (Rivest, Shamir, Adelson algorithm). Come lo implementiamo?

- e chiave pubblica
- d chiave privata
- n prodotto di numeri primi

Sapendo che nella aritmetica modulare possiamo fare

$$(a \bmod n)^d \bmod n = a^d \bmod n$$

Quindi per cifrare facciamo

$$c = m^e \bmod n$$

e per decifrare

$$c^d \bmod n = m$$

e otteniamo

$$\begin{aligned} & \underbrace{(m^e \bmod n)}_c^d \bmod n \\ & m^{ed} \bmod n \\ & \rightarrow \boxed{m \equiv_n m^{ed}} \end{aligned}$$

Come scegliamo le chiavi?

- Presi p e q primi da 1024bit
- Calcoliamo $n = pq$, $z = (p-1)(q-1)$
- Scegliamo $e (< n)$ tale che $\text{mcd}(e, z) = 1$
- Scegliamo d tale che $ed \equiv_z 1$ (ovvero $ed - 1$ sia divisibile per z)
 $\rightarrow$ Abbiamo ora chiave pubblica $K_B^+ = (n, e)$ e chiave privata $K_B^- = (n, d)$

Possiamo anche notare la proprietà

$$K_B^-(K_B^+) = m = K_B^+(K_B^-)$$

Vediamo quanto è sicuro la RSA. Supponiamo di conoscere la chiave pubblica (n, e) dobbiamo trovare ora la chiave privata (n, d) . Quanto è difficile trovare d ?

$$d \equiv_{(p-1)(q-1)} e^{-1}$$

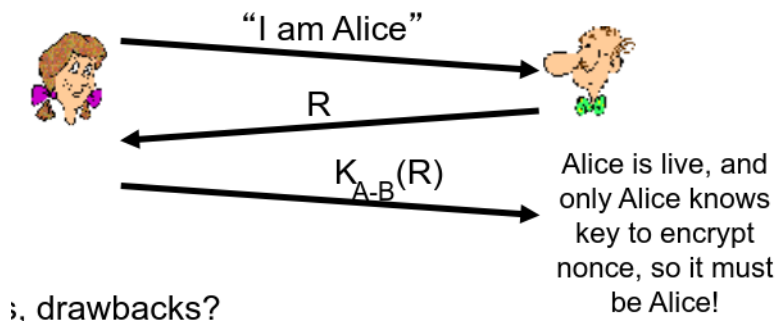
Quindi dobbiamo trovare p e q partendo da $n = pq$. Qui dovremmo risolvere dei problemi di fattorizzazione su numeri grandi numeri (1024bit / 617 cifre decimali). Il costo computazionale richiederebbe miliardi di anni.

Nella realtà calcolare RSA ad ogni connessione sarebbe computazionale costoso (per colpa del calcolo delle potenze), quindi facciamo RSA per condividere una chiave comune. Poi facciamo connessione a chiave simmetrica.

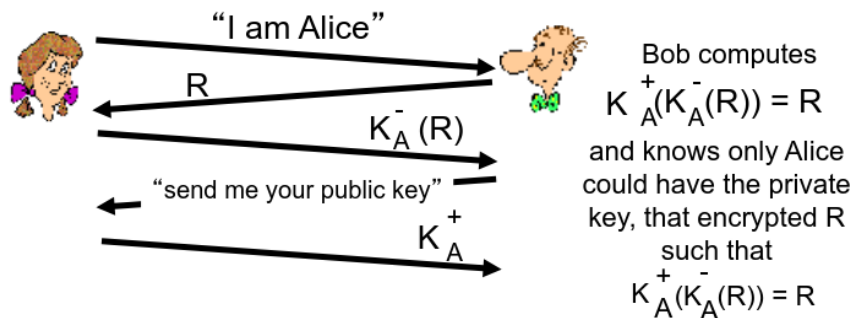
3.2 Autenticazione

Come proviamo la nostra identità tra client e server?

- ap 1.0: Io sono x
- ap 2.0: IP + Io sono x
- ap 3.0: IP + password + Io sono x
- ap 3.1: IP + encrypted password + Io sono x
- ap 4.0: scegliamo numero **nonce** (*once-in-a-lifetime*) R , client dice "Io sono x ", il server manda R al client, il client risponde con $K(R)$ criptato con chiave comune. Il problema è che serve chiave condivisa



- ap 5.0: Io sono x , server manda once R , il client risponde con la criptazione su chiave privata $K_A^-(R)$, il server chiede di mandargli la chiave pubblica K_A^+ e controlla se $R = K_A^+(K_A^-(R))$.



Security hole: l'attaccante posa come server al client e come client al server.

3.2.1 Digital signatures

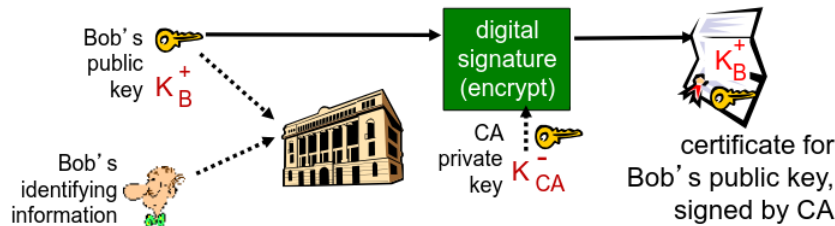
Il sender spedisce il messaggio m criptandolo con la sua chiave privata K_B^- creato il messaggio firmato $K_B^-(m)$. Così il ricevente capisce se lo ha mandato il client facendo $K_B^+(K_B^-(m)) = m$. *Il problema?* Fare public-key-encrypt su lunghi messaggi è computazionalmente costoso.

- Hash function: applichiamo H su m e otteniamo il messaggio di lunghezza fissa $H(m)$. Poi dopo applichiamo la criptazione.
→ **Digital signature:** hashare un messaggio e poi criptarlo/decriptarlo su chiave privata e pubblica.

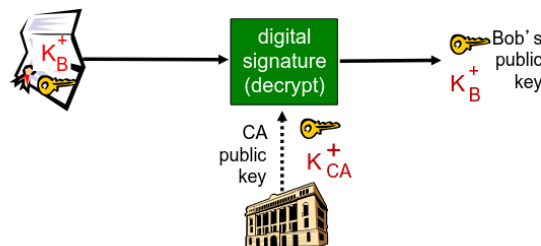
Soffre sempre di man in the middle (ap 5.0).

3.2.2 Certification authority

Una entità E (persona, router, ecc) registra la sua chiave pubblica con CA, E prova la sua identità al CA, CA collega E alla sua chiave pubblica.



Il ricevente come ottiene la chiave pubblica del mandante? Ottiene il certificato dal client e si applica la chiave pubblica del CA sul certificato

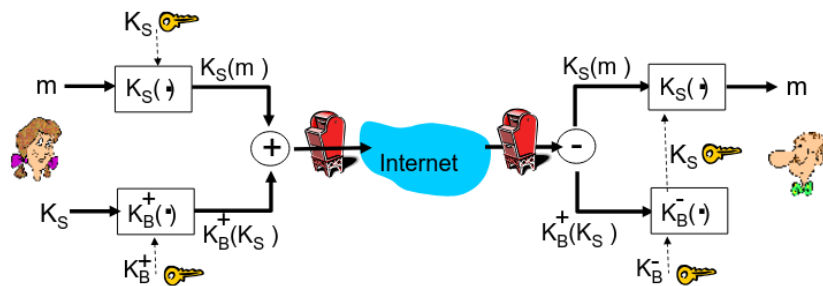


Ovvero facciamo $K_B^+ = K_{CA}^-(K_{CA}^+(K_B^+))$ così mandiamo in modo sicuro la chiave pubblica del client. (Es. TLS per HTTPS)

3.3 Securing e-mail

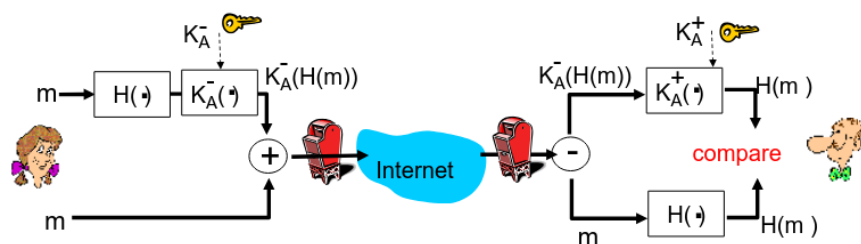
Come mandiamo una mail m sicura?

- Chiave simmetrica casuale K_S
- Spediamo il messaggio criptato $K_S(m)$ e la chiave pubblica criptata $K_S(K_B^+)$
- Per decriptarlo si utilizza la chiave privata per ottenere K_S
- Utilizziamo K_S per decriptare m

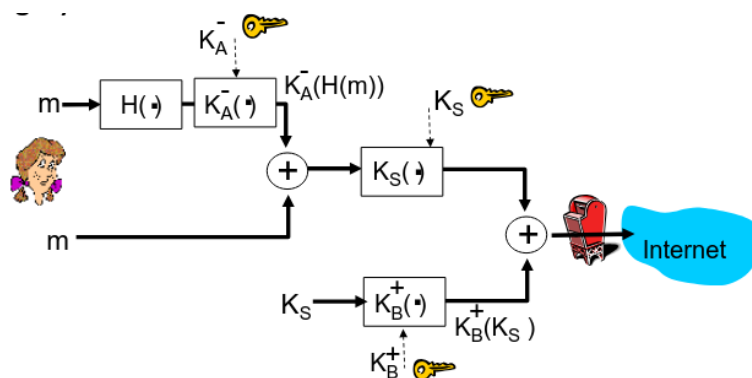


$$K_B^-(K_B^+(K_S)) = K_S$$

Autenticazione online (non sicura) Spediamo il messaggio con Hash condiviso $H(m)$ + criptazione su chiave pubblica/privata.



Autenticazione online (sicura) Spediamo messaggio su hash comune $H(m)$, criptato su chiave privata sender + chiave simmetrica K_S su chiave pubblica receiver $K_S(K_B^+)$.



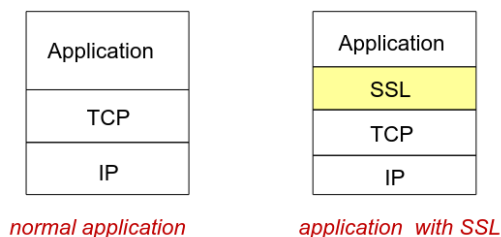
$$K_B^-(K_B^+(K_S)) = K_S$$

$$K_S(K_A^-(H(m)) + m) = K_A^-(H(m)) + m$$

$$K_A^+(H(m)) = H(m) \text{ da confrontare con la nostra } H \text{ t.c. } = H(m)$$

Il receiver deve conoscere la sua chiave privata, la chiave pubblica del sender e la hash function che è pubblica.

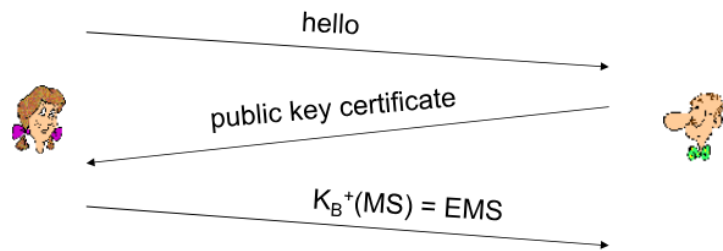
3.4 Securing TCP connections: SSL



SSL offre una API alle applicazioni (nella maggior parte dei LP).

Potremmo fare qualcosa come per autenticazione online (sicura) ma ci serve uno stream di dati costante.

Attraverso un **handshake** possiamo autenticare il sender e receiver.



- MS: master secret
- EMS: encrypted master secret

Ovviamente è cattiva pratica utilizzare la stessa chiave per più di 1 operazione di criptazione, si utilizzano diverse chiavi **MAC** (message authentication code)

- K_C : chiave di criptazione client to server
- M_C : chiave MAC client to server
- K_S : chiave di criptazione server to client
- M_S : chiave MAC server to client

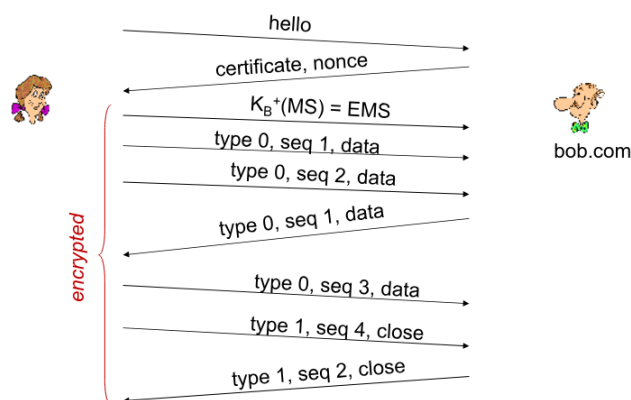
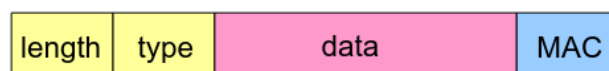
Via TCP come suddividiamo il pacchetto fra data e MAC?



Come rendiamo sicuro il MAC in modo che se qualcuno lo sniffi non lo posso riutilizzare?

$$MAC = \langle type, sequence, data \rangle$$

con *type* che indica la tipologia di pacchetto (mess, handshake, chiusa conn., ecc)



Questo modello di SSL semplificato non basta, tipo se volessimo cambiare protocollo di criptazione? Specificare il protocollo scelto, ecc?

Comuni modelli di SSL simmetrici cifrai

- DES
- 3DES
- RC2
- RC4

Comuni modelli di SSL su chiave pubblica/privata

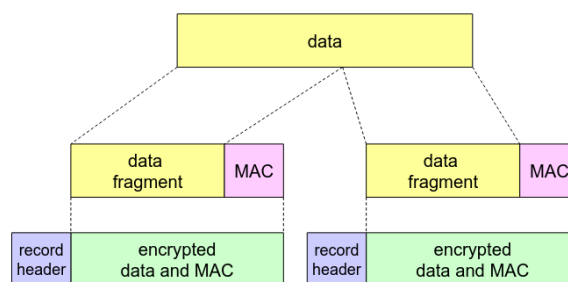
- RSA

3.4.1 Real SSL

Attraverso l'**handshake** possiamo fare autenticazione server, negoziazione su scelta del algoritmo di crittazione, stabilire le chiavi e opzionalmente autenticazione client.

- client manda la lista di algoritmi di crittazione supportati
- il server risponde: choice algoritmo + certificato + server nonce
- client controlla il certificato, estrae chiave pubblica server, genera pre_master_secret, la critta con chiave pubblica server e glielo spedisce
- client e server indipendentemente calcolano decriptazione e MAC key dalle pre_master_secret e nonce
- client manda MAC di tutti gli handshake
- server manda MAC di tutti gli handshake

Protocollo:

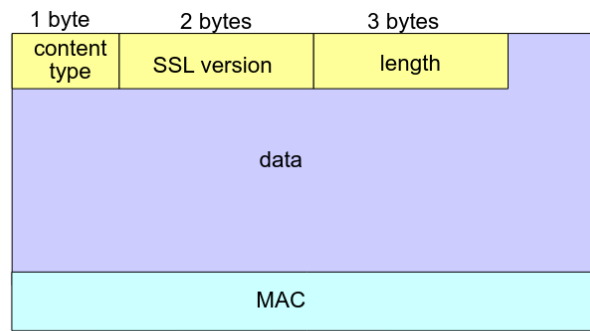


record header: content type; version; length

MAC: includes sequence number, MAC key M_x

fragment: each SSL fragment 2^{14} bytes (~16 Kbytes)

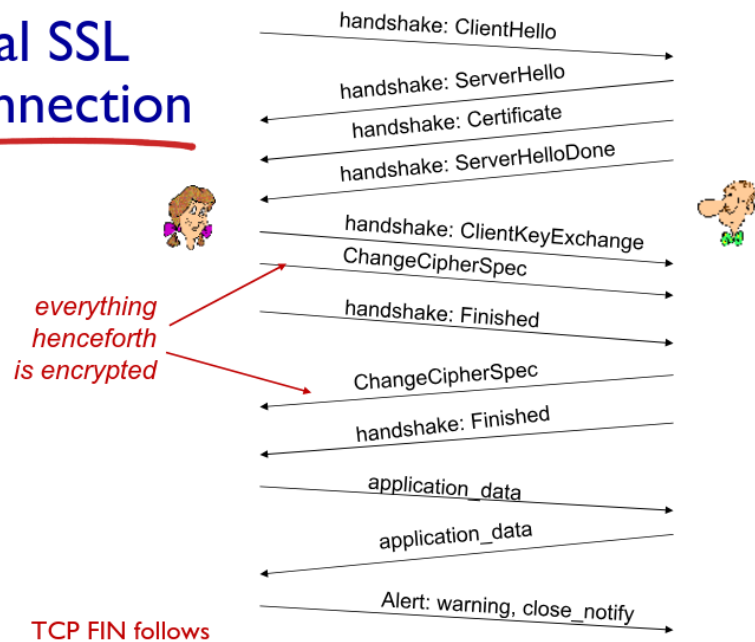
Formato del pacchetto:



data and MAC encrypted (symmetric algorithm)

Connessione:

Real SSL connection



3.5 Network layer security: IPsec

Come rendiamo sicura la trasmissione tra due reti?

3.5.1 VPN

Criptiamo prima di andare in internet pubblico, attraverso la logica separata dal resto del traffico.

Il sender e reciver sono pubblici (header) mentre il traffico diventa criptato.

Alla base delle VPN troviamo **IPsec** ovvero un insieme di protocolli per proteggere comunicazioni su rete IP cifrando i dati che viaggiano tra due dispositivi.

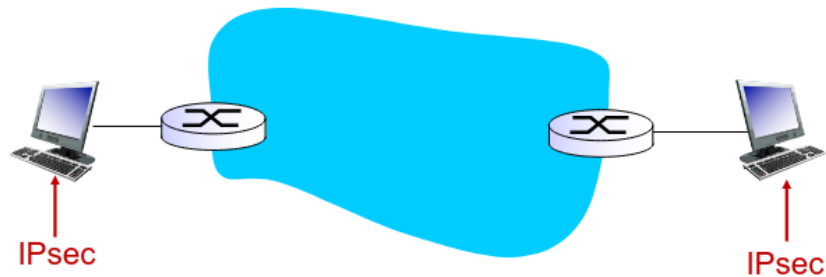
- Confidenzialità (dati cifrati)
- Integrità (dati non possono essere modificati senza accorgersene)
- Autenticazione (di chi sta parlando)
- Anti-replay (impedisce il riutilizzo dei pacchetti in attacchi)

Per implementarlo abbiamo due opzioni:

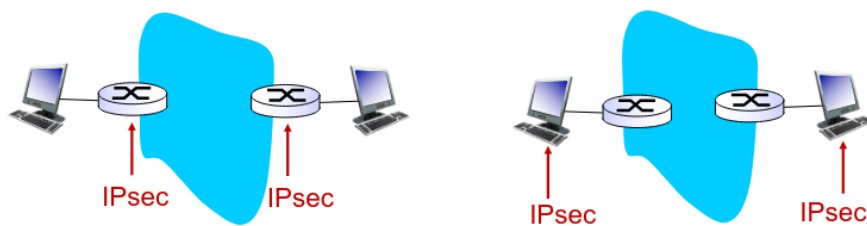
- **AH** (Authentication Header): Garantisce autenticazione e integrità, non cifra i dati e non garantisce confidabilità
- **ESP** (Encapsulating Security Payload): Garantisce cifratura + autenticazione + confidabilità

Modalità di funzionamento (Ipssec lavora a livello rete)

- **Transport Mode:** i client si occupano del IPsec



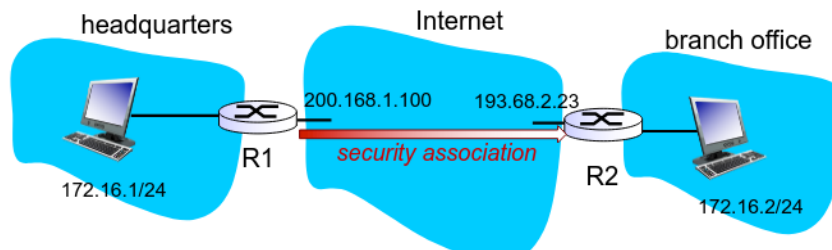
- **Tunneling Mode:** i router si occupano del IPsec



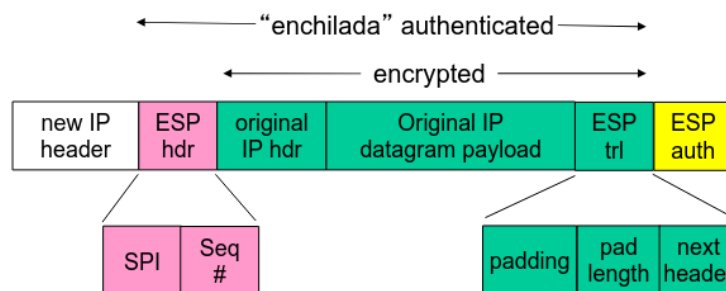
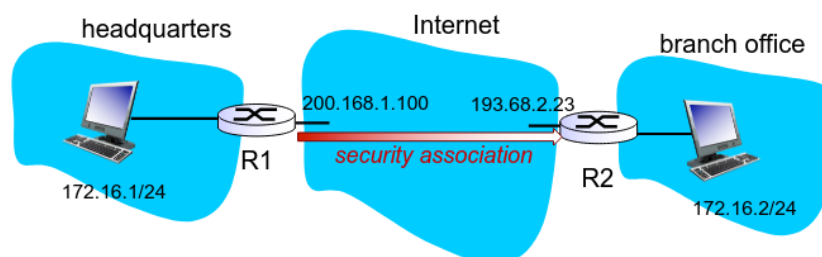
Host mode with AH	Host mode with ESP
Tunnel mode with AH	Tunnel mode with ESP

most common and
most important

Security associations (SAs) Prima di iniziare a spedire i dati dobbiamo fare "security associations" fra sender e receiver. Una SA è l'insieme di parametri che stabiliscono algoritmi, chiavi e regole per cifrare/autenticare i pacchetti.



Security Association Database (SAD) Il Security Association Database (SAD) è la struttura dati in cui un sistema che usa IPsec salva tutte le Security Associations (SA) attive.



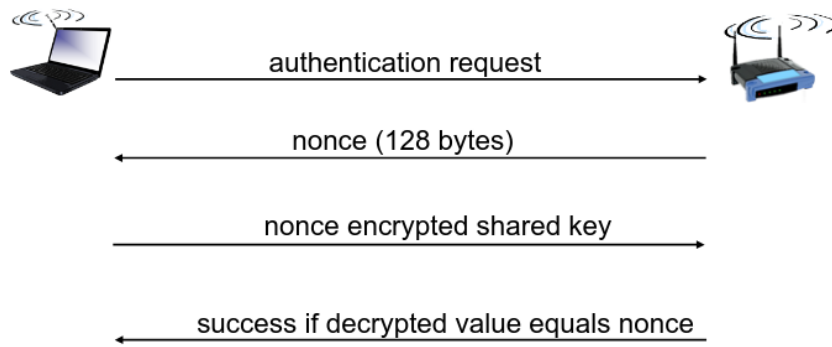
- ESP trailer: bit padding per arrivare a una certa dimensione
- ESP header:
 - SPI, so receiving entity knows what to do
 - Sequence number, to thwart replay attacks
- MAC in ESP auth field is created with shared secret key

Security Policy Database (SPD) Il Security Policy Database (SPD) è la struttura dati che contiene le regole di sicurezza che stabiliscono quale traffico IP deve essere protetto e come. SPD indica cosa se e come proteggere un pacchetto, SAD indica come farlo (chiavi e parametri).

3.6 Securing wireless LANs

WEP: stabilire rete temporanea tra due dispositivi wifi. Molto efficiente da implementare via hw o sw.

Sicuro come modello matematico (criptazione 802.11), non sicuro perchè durante la security association l'header dei pacchetti (criptato con la stessa chiave WEP) è noto e quindi per analisi di correlazione tra i pacchetti si riesce a fare bruteforce per decrittare i pacchetti.

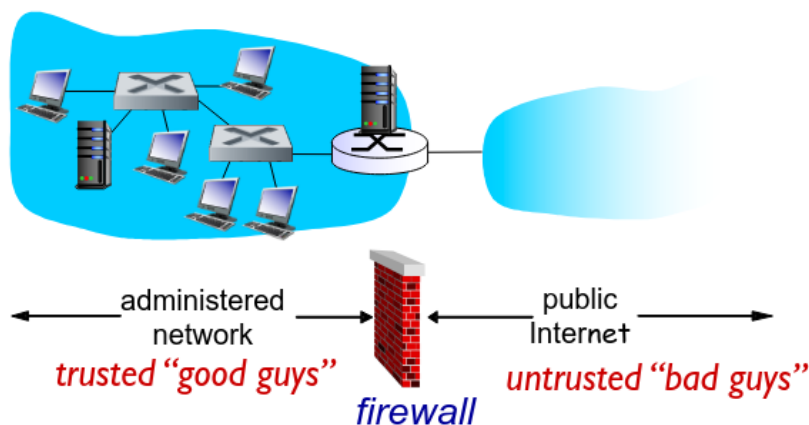


802.11i Migliorata la sicurezza attraverso più forme di criptazioni migliori, key distribution, authentication server separato da authentication server.

3.7 Operational security: firewalls and IDS

3.7.1 Firewall

Isola una rete locale da Internet, accetta solo certi pacchetti che possono entrare mentre altri li blocca



I router filtrano pacchetto per pacchetto per fare il forward/drop.

Access Control Lists Tabella di regola top-bottom per i pacchetti.

action	source address	dest address	protocol	source port	dest port	flag bit
allow	222.22/16	outside of 222.22/16	TCP	> 1023	80	any
allow	outside of 222.22/16	222.22/16	TCP	80	> 1023	ACK
allow	222.22/16	outside of 222.22/16	UDP	> 1023	53	---
allow	outside of 222.22/16	222.22/16	UDP	53	> 1023	----
deny	all	all	all	all	all	all

- Stateless packet filter: guarda ogni pacchetto singolarmente

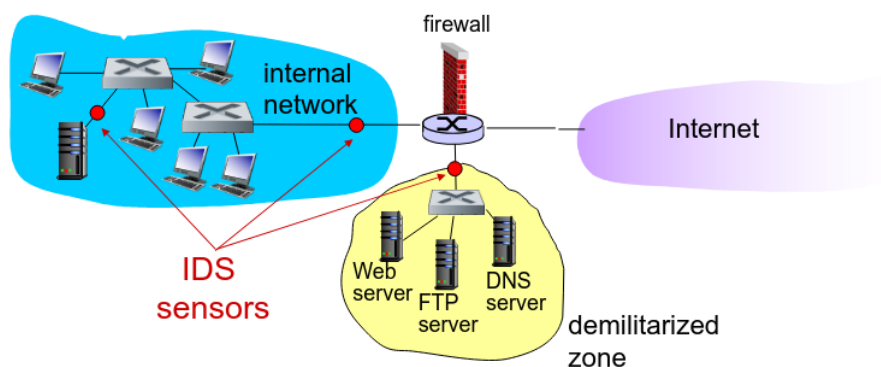
- Stateful packet filter: tiene traccia delle connessioni

action	source address	dest address	proto	source port	dest port	flag bit	check connexion
allow	222.22/16	outside of 222.22/16	TCP	> 1023	80	any	
allow	outside of 222.22/16	222.22/16	TCP	80	> 1023	ACK	X
allow	222.22/16	outside of 222.22/16	UDP	> 1023	53	---	
allow	outside of 222.22/16	222.22/16	UDP	53	> 1023	----	X
deny	all	all	all	all	all	all	

Application gateways Analizza il traffico capendo il protocollo applicativo (HTTP, FTP, SMTP, ecc.), per fare filtraggio avanzato. Un Application Gateway non inoltra semplicemente i pacchetti, ma analizza url, bloccare certe richieste, leggere header e cookies, ecc.

3.7.2 IDS

Intrusion detection system attraverso deep packet inspection e examine correlation su molteplici pacchetti.



4 Data Plane

4.1 Network layer

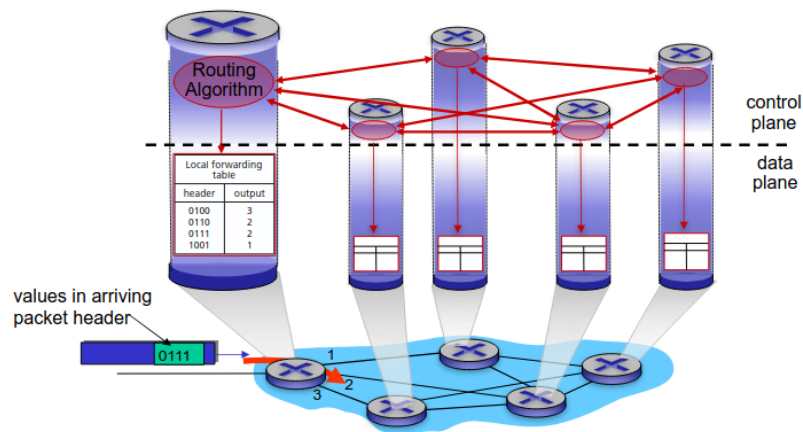
Il network layer si occupa del segmento dal sender al receiver.

- **Forwarding**: processo locale con cui un router inoltra ogni pacchetto dall'ingresso all'uscita corretta usando la tabella di inoltro.
- **Routing**: processo globale che determina i percorsi nella rete e costruisce le tabelle usate dai router.

4.1.1 Data plane

Funzione locale di ogni router che inoltra i datagrammi dall'ingresso all'uscita corretta (forwarding).

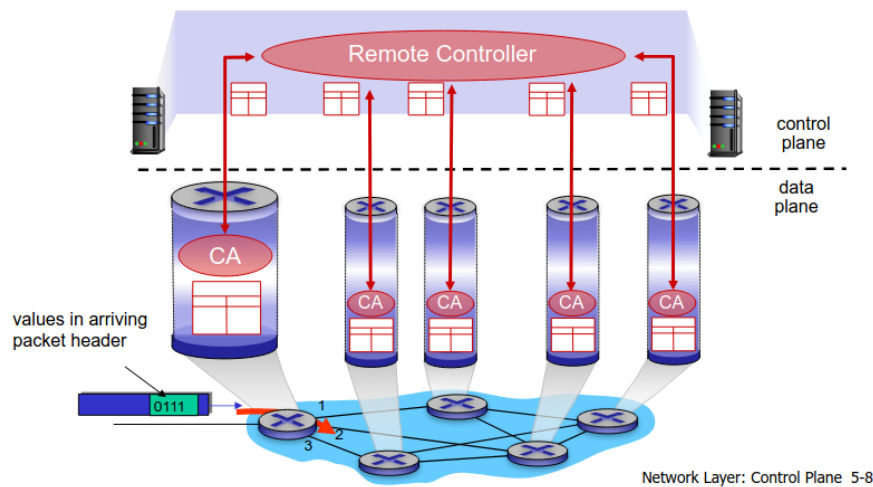
Individual routing algorithm components *in each and every router* interact in the control plane



4.1.2 Control plane

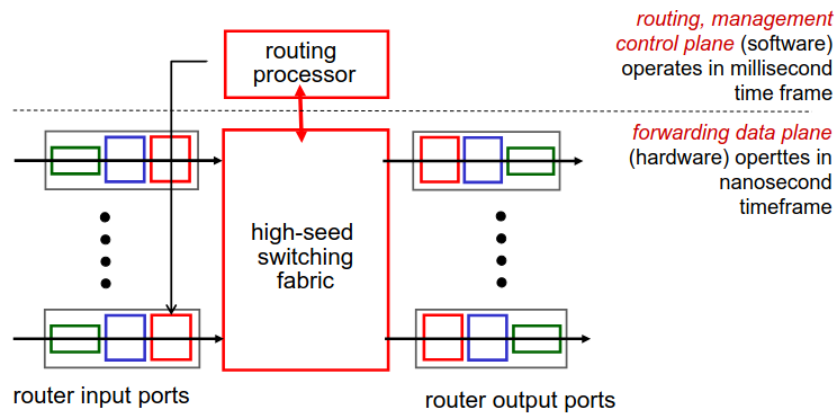
Logica globale della rete che decide i percorsi dei datagrammi tra sorgente e destinazione (routing, via algoritmi tradizionali o SDN).

A distinct (typically remote) controller interacts with local control agents (CAs)



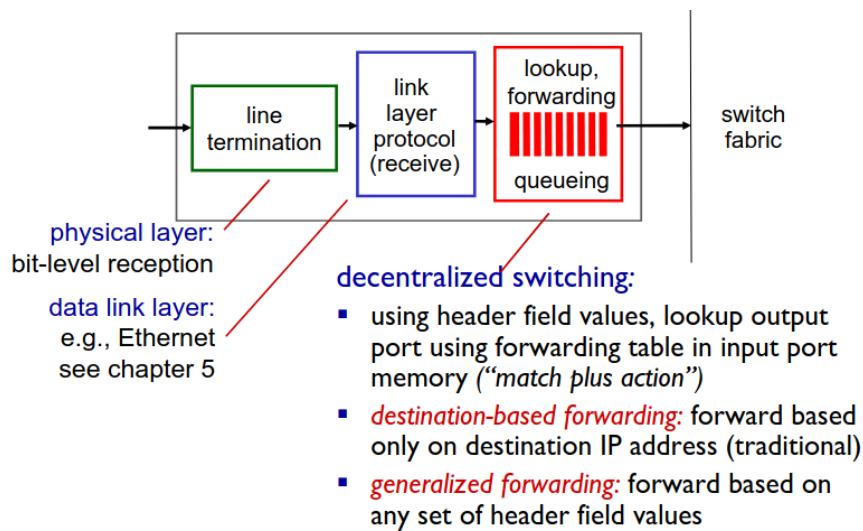
4.2 Come e' fatto un router

Visione ad alto livello della architettura di un router



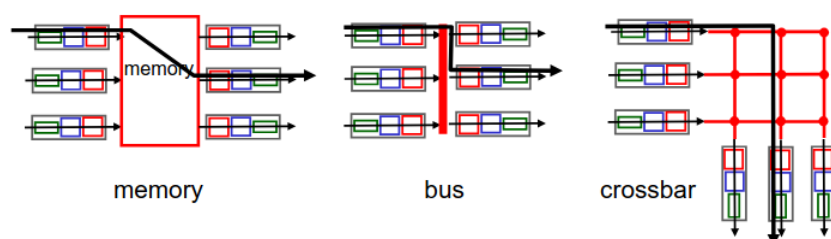
4.2.1 Input

In particolare gli input sono



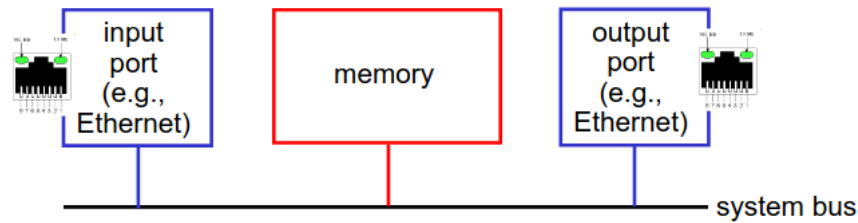
Switching fabrics

Trasferire un pacchetto dal input buffer ad un output buffer



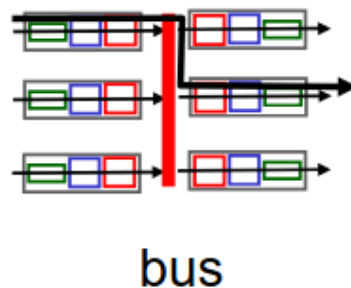
Switching via memory

I pacchetti vengono copiati nella memoria (limitato da CPU e memory speed)



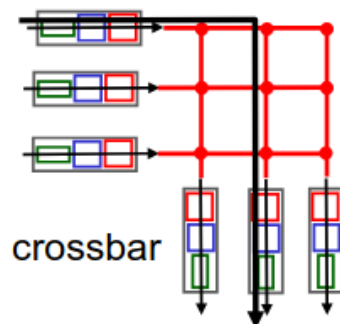
Switching via a bus

Pacchetti da input memory al output memory via un bus condiviso.



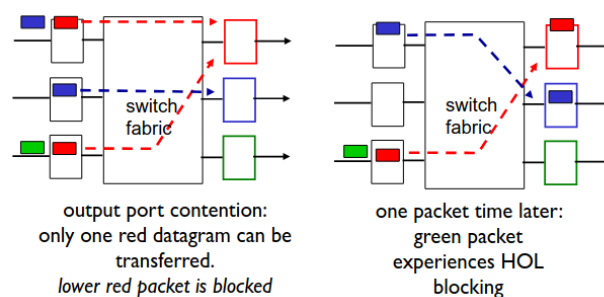
Switching via interconnection network

Un crossbar è una rete interna al router che collega direttamente ogni porta di ingresso con ogni porta di uscita. (matrice, condivisione dei pacchetti in parallelo)

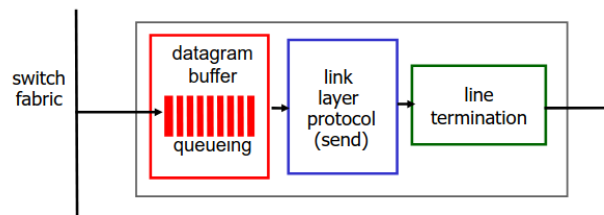


Input port queuing

Head-of-the-Line (HOL) blocking: è quando il pacchetto in testa alla coda blocca tutti gli altri dietro, anche se potrebbero essere inoltrati verso uscite libere.



4.2.2 Output



- **Buffering** (packets possono essere persi per colpa della congestione se non abbiamo un buffer)

Possiamo fare buffering quando il *arrival rate* supera la *line speed* dello switch. Questo puo' portare loss per colpa del delay e in caso di buffer overflow.

Quanto buffer? RFC 3439 dice: " $RTT \times C$ " (con C come link capacity). Es. 10Gpbs link $\Rightarrow$ 2.5Gbit buffer.

O se no possiamo fare generalmente

$$\text{buffer} = \frac{RTT \times C}{\sqrt{N}}$$

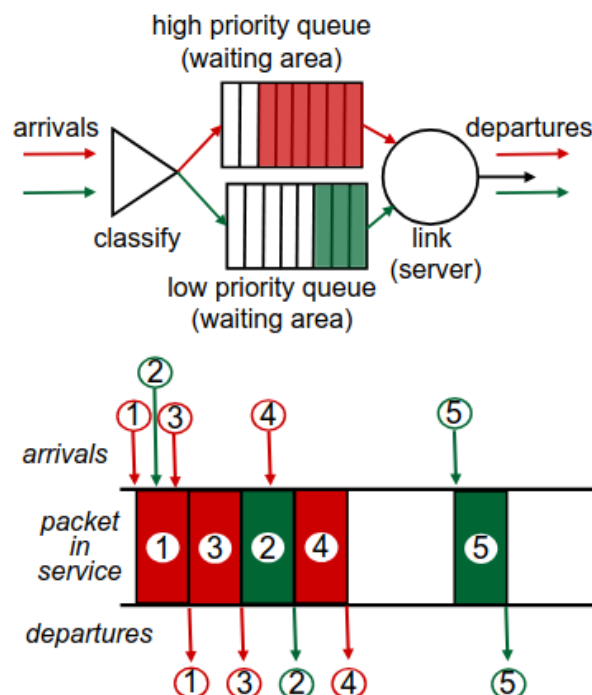
- **Scheduling** (next packet to send on link; prioritá' per ottimizzare le performance)
Quale meccanismo di scheduling scegliamo?

- **FIFO**

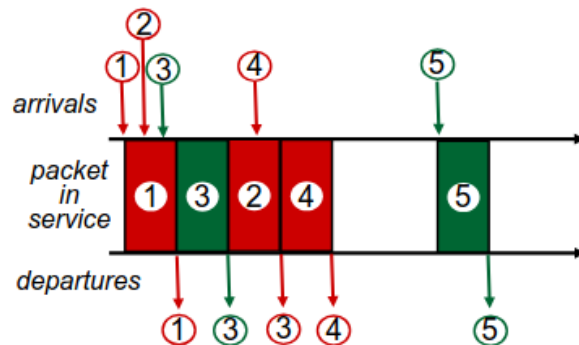
- **Discard policy**

- * Tail drop

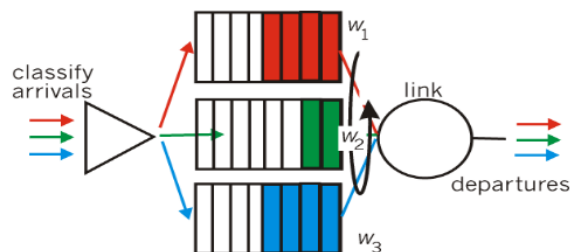
- * **Priority**: send highest priority queued packet, supporta multiple classes, with different priorities



- * Random
- **Round Robin**: cyclically scan class queues, sending one complete packet from each class

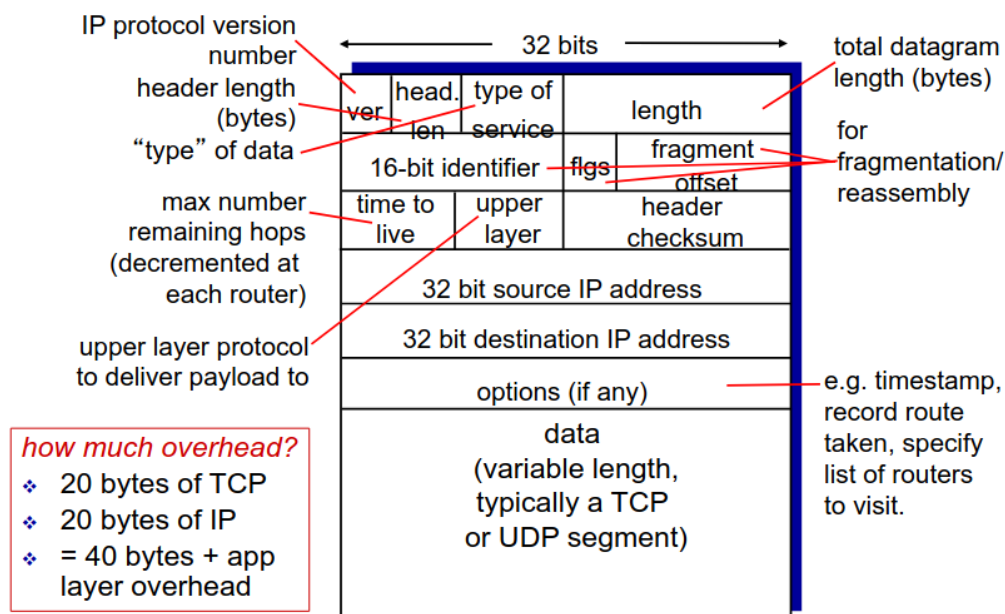


- **Weighted Fair Queuing (WFQ)**: E' una generalizzazione del R.R., each class gets weighted amount of service in each cycle



4.3 Internet Protocol

4.3.1 Datagram format



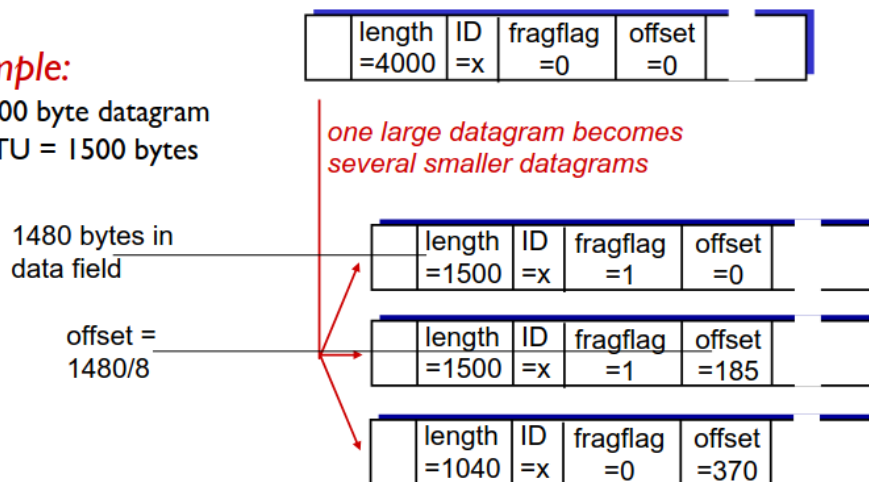
4.3.2 Fragmentation

- network links have MTU (max.transfer size) - largest possible link-level frame

- large IP datagram divided (“fragmented”) within net
 - one datagram becomes several datagrams
 - “reassembled” only at final destination
 - IP header bits used to identify, order related fragments

example:

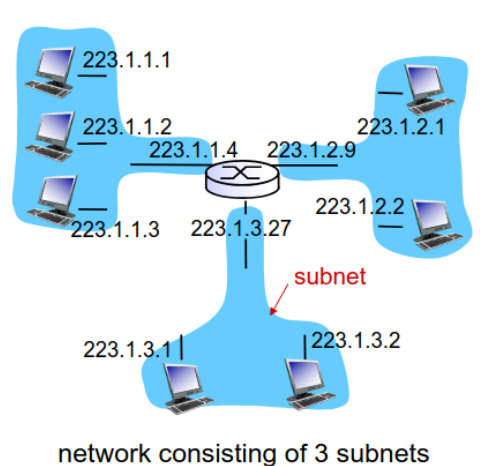
- ❖ 4000 byte datagram
- ❖ MTU = 1500 bytes



4.3.3 IPv4 addressing

- IP address: 32 bit per host, router, interface
- Interface: connection between host/router and physical link

Subnets Device interfaces with same subnet part of IP address.
Can physically reach each other without intervening router

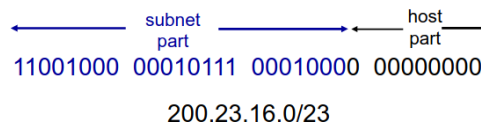


To determine the subnets, detach each interface from its host or router, creating islands of isolated networks.
Each isolated network is called a subnet.

IP addressing: CIDR

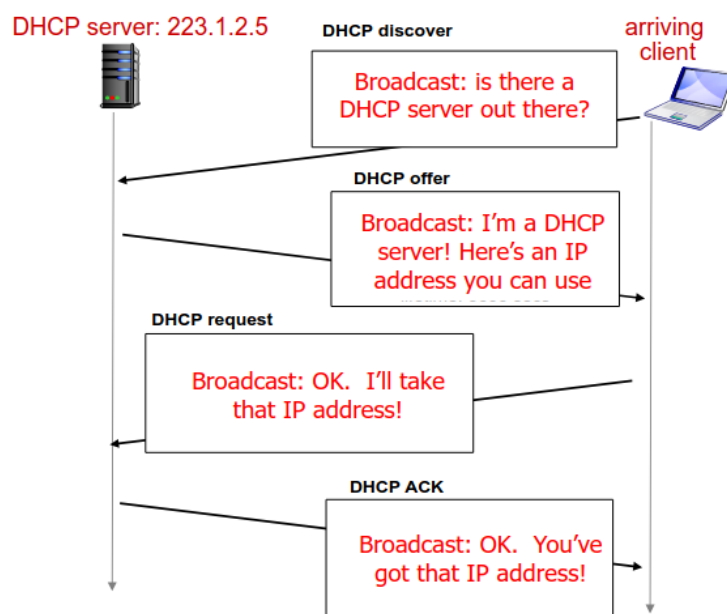
CIDR: Classless InterDomain Routing

- Subnet portion of address of arbitrary length
- Address format: a.b.c.d/x, where x is #bits in subnet portion of address



IP addresses: how to get one?

- **Static** (hard coded in OS)
- **DHCP**: Dynamic Host Configuration Protocol: dynamically get address from as server
 - host broadcasts “DHCP discover” msg [optional]
 - DHCP server responds with “DHCP offer” msg [optional]
 - host requests IP address: “DHCP request” msg
 - DHCP server sends address: “DHCP ack” msg



DHCP can return more than just allocated IP address on subnet:

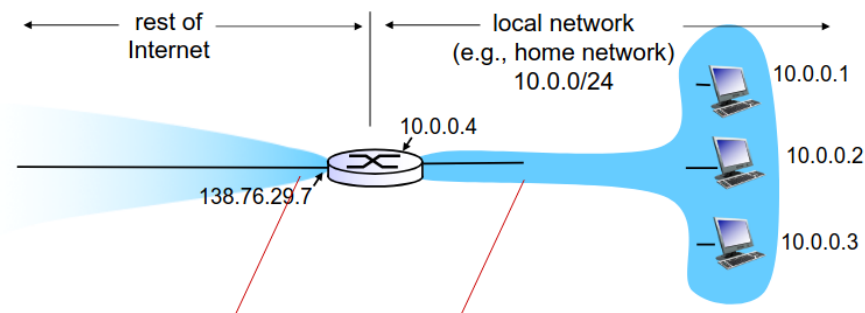
- address of first-hop router for client
- name and IP address of DNS sever
- network mask

How does an ISP get block of addresses?

ICANN: Internet Corporation for Assigned

- allocates addresses
- manages DNS
- assigns domain names, resolves disputes

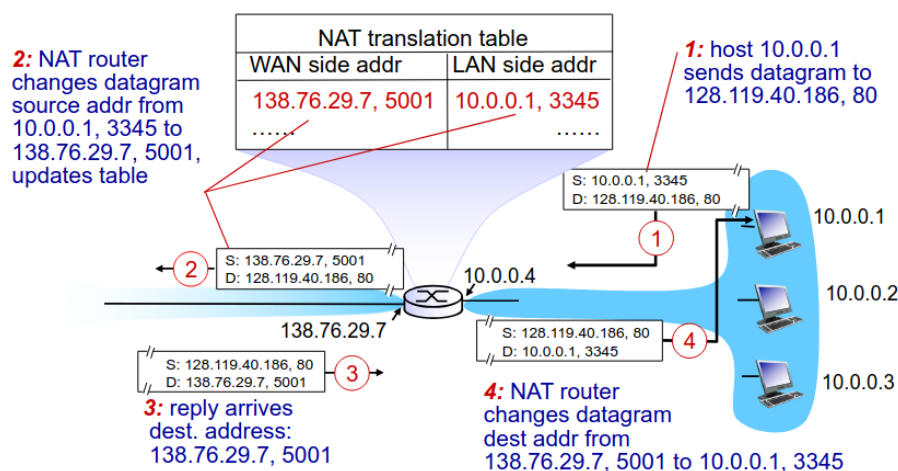
4.3.4 Network address translation



Motivation: local network uses just one IP address as far as outside world is concerned.

NAT router must:

- **outgoing datagrams:** replace (source IP address, port #) of every outgoing datagram to (NAT IP address, new port #)
- **remember (in NAT translation table)** every (source IP address, port #) to (NAT IP address, new port #) translation pair
- **incoming datagrams:** replace (NAT IP address, new port #) in dest fields of every incoming datagram with corresponding (source IP address, port #) stored in NAT table



4.3.5 IPv6

Initial motivation: 32-bit address space soon to be completely allocated.

Extra: header format helps speed processing/forwarding and header changes to facilitate QoS.

IPv6 datagram format

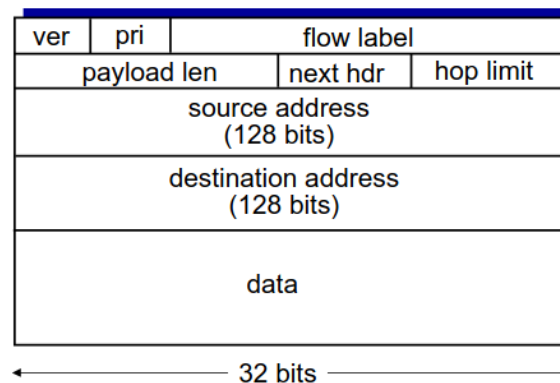
- 40byte fixed header
- no fragmentation allowed

priority: identify priority among datagrams in flow

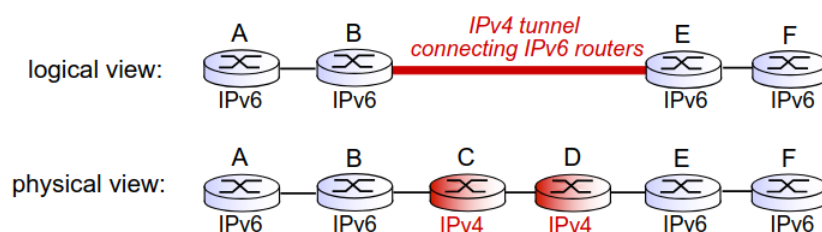
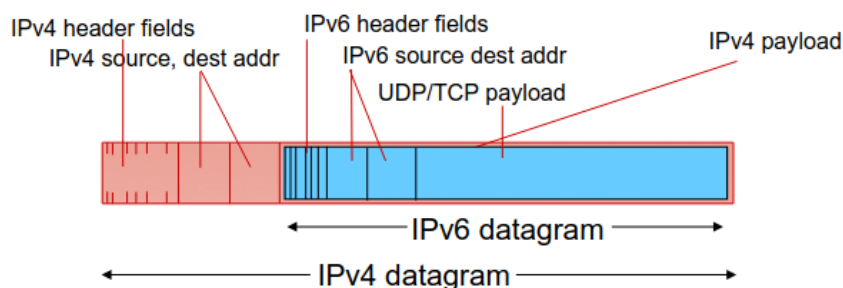
flow Label: identify datagrams in same “flow.”

(concept of “flow” not well defined).

next header: identify upper layer protocol for data

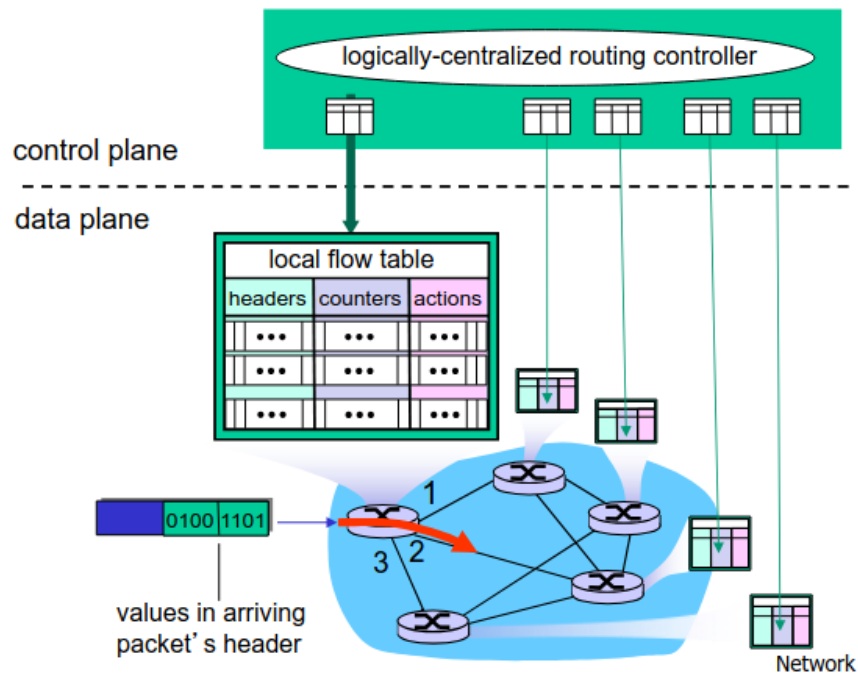


How will network operate with mixed IPv4 and IPv6 routers? **Tunneling!** IPv6 datagram carried as payload in IPv4 datagram among IPv4 routers



4.4 Generalized Forward and SDN

Each router contains a flow table that is computed and distributed by a logically centralized routing controller



4.4.1 OpenFlow data plane abstraction

- flow: defined by header fields
- generalized forwarding: simple packet-handling rules
 - Pattern: match values in packet header fields
 - Actions: for matched packet: drop, forward, modify, matched packet or send matched packet to controller
 - Priority: disambiguate overlapping patterns
 - Counters: #bytes and #packets

5 Control Plane